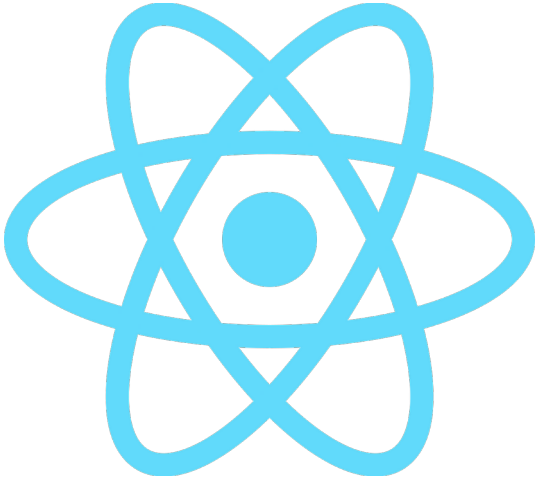
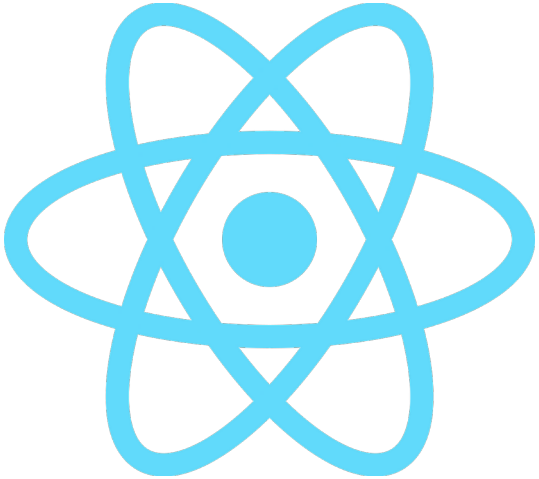


React

Making components interactive — state, props and data flow

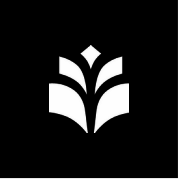
UEF// University of Eastern Finland

Course materials originally by Sonsoles López-Pernas, adapted by Maria Kuismin



React

Student is able to
analyze user interface requirements and design reusable, component-based web applications using React



Contents

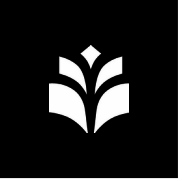
Props — making components configurable
- What are props, and how do they make components configurable?
- How are callback functions passed as props?

State — storing and tracking changing data
- What is state, and how does it differ from props?
- How does useState work, and where should state be owned?

Events and forms — responding to user interaction
- How do React components respond to user interactions?
- How are controlled forms implemented in React?

Side effects and lifecycle — useEffect
- What is the difference between useState and useEffect?
- When and how is useEffect used?

Component libraries — ready-made React components
- What are React component libraries, and how are they used?



- Props — making components configurable
  - What are props, and how do they make components configurable?
  - How are callback functions passed as props?
- State — storing and tracking changing data
  - What is state, and how does it differ from props?
  - How does useState work, and where should state be owned?
- Events and forms — responding to user interaction
  - How do React components respond to user interactions?
  - How are controlled forms implemented in React?
- Side effects and lifecycle — useEffect
  - What is the difference between useState and useEffect?
  - When and how is useEffect used?
- Component libraries — ready-made React components
  - What are React component libraries, and how are they used?

Properties customize each component instance

- Props are the properties of each instance of a component
- The components can be customized with different parameters when they are created . These creation parameters are called props.
- Props are data in Javascript provided as attributes to the component at rendering time. They can be anything type , string, number, boolean, object, ...
- Props are IMMUTABLE, read- only
- A component never has to modify its own props

Passing props in practice

- Props are passed as attributes when rendering the component
- Accessed inside the component as props.name, props.available, etc.
- Different instances of the same component can have different values
- Props enable reusability: one component, many variations

```
function PostCard(props) {
  return (
    <div>
      <h3>{props.title}</h3>
      <p>{props.caption}</p>
      <p>❤️ {props.likes} likes</p>
    </div>
  );
}

<PostCard
  title="Toast's morning nap"
  caption="Sleeping in a sunbeam"
  likes={47}
/>
<PostCard
  title="Naranja on the windowsill"
  caption="Watching birds again"
  likes={23}
/>
```

Optional values and missing props

```jsx
function PostCard({
  title,
  caption,
  // default values if not provided
  likes = 0,
  imageUrl = null,
  imageAlt = ""
}) {
  return (
    <div>
      // render image only if imageUrl is
provided
      {imageUrl && (
        <img src={imageUrl} alt={imageAlt} />
      )}
      <h3>{title}</h3>
      <p>{caption}</p>
      <p>❤️ {likes} likes</p>
    </div>
  );
}
```

```jsx
// All props provided:
<PostCard
  title="Toast's morning nap"
  imageUrl="toast.jpg"
  imageAlt="Orange cat sleeping on a white
pillow"
  caption="Sleeping in a sunbeam"
  likes={47}
/>

// imageUrl and imageAlt missing
// — no image rendered:
<PostCard
  title="Naranja on the windowsill"
  caption="Watching birds again"
  likes={23}
/>
```

Passing functions as props — callback props

Props can also be functions — called callback props. Used when a child component needs to notify the parent.
This is one-way data binding in practice: data flows DOWN (title, id), events flow UP (onDelete callback)

```
function PostCard(props) {
  return (
    <div>
      <h2>{props.title}</h2>
      <button onClick={() => props.onDelete(props.id)}>   // callls the function passed from parent
        Delete
      </button>
    </div>
  );
}

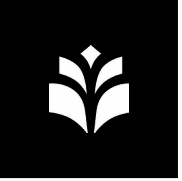
// In parent (App):
<PostCard
  title="Toast's morning"
  id={1}
  onDelete={(id) => deletePost(id)} // parent passes the function down as a prop
/>
```

Summarizing props

Props (properties) make components configurable:
→ passed as attributes when rendering
→ read-only inside the component
→ can be any JavaScript value: string, number, object, function

Callback props make events flow up:
→ parent passes a function down as a prop
→ child calls it when something happens
→ parent decides what to do with the event

Next: what if a component needs to store something? → state



Props — making components configurable

- What are props, and how do they make components configurable?
- How are callback functions passed as props?

State — storing and tracking changing data

- What is state, and how does it differ from props?
- How does useState work, and where should state be owned?

Events and forms — responding to user interaction

- How do React components respond to user interactions?
- How are controlled forms implemented in React?

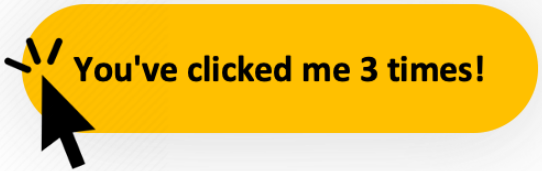
Side effects and lifecycle — useEffect

- What is the difference between useState and useEffect?
- When and how is useEffect used?

Component libraries — ready-made React components

- What are React component libraries, and how are they used?

Components with state - ButtonCounter



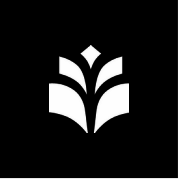

```jsx
import React, { useState } from 'react';

function ButtonCounter() {
  const [clicks, setClicks] = useState(0);

  const handleClick = () => {
    setClicks(clicks + 1);
  };

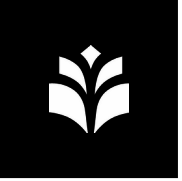
  return (
    <Button
      onClick={handleClick}
      text={`You've clicked me ${clicks} times!`}
    />
  );
}

export default ButtonCounter;
```



State: information that changes over time

- state is additional information about the component that is not provided at the time of rendering but rather created and updated by the component
- useful when component has to maintain certain information between renderings
- state is mutable, but cannot be modified directly
- the main feature of the state is that the component is repainted each time that the state changes
- normally, a state is something that we would need to be able to restore our application to the exact same “state” (freeze snapshot)
- important to distinguish between application state and component state



Two levels of state: app and component

- **Application state:** Keep all the data together that I would need to save in a database or send/retrieve from API (e.g., gameplay status, shopping cart). Normally it's the state of the root component (<App/>)
- **Component state:** Needed to implement some aspects in the UI that we do not need to save/send anywhere (e.g., accordion open/closed, dropdown displayed/hidden, unsaved form fields)

React hooks: useState and useEffect

- Hooks are functions that allow you to “hook” the React state and lifecycle from function components
- Hooks allow us to update the state and perform actions when changes occur in the component lifecycle
- The most used Hooks are useState and useEffect

useState for primitive values - Counter

- When calling useState we get a pair of two values: the first one is the state, and the second one is a function that allows us to update the state.
- The argument that useState takes is the initial state (used for the first render)

```
export default function Counter() {
    // Declare a new state variable called "count"
    const [count, setCount] = useState(0);

    return (
        <div>
            <p>You clicked {count} times</p>
            <button onClick={() => setCount(count =>
count + 1)}>
                Click me
            </button>
        </div>
    );
}
```

Every time the button is clicked, it calls the setCount function that allows to modify the state (in this case, by increasing 1)

useState for objects - CatProfile

```
import { useState } from "react";

export default function CatProfile() {
    const [cat, setCat] = useState({
        name: "Toast",
        age: 6,
        color: "orange",
        isAsleep: true
    });

    return (
        <>
            <h1>{cat.name}</h1>
            <p>
                {cat.color} cat, {cat.age} years old.
            </p>
            <p>{cat.isAsleep ? "😴 Sleeping" : "😺 Awake"}</p>
            <button onClick={() => setCat({...cat, isAsleep: !cat.isAsleep})}>
                {cat.isAsleep ? "Wake up" : "Go to sleep"}
            </button>
        </>
    );
}
```

State must be updated via setter function

```
const [count, setCount] = useState(0);
```

Incorrect: `count = count + 1;`
JavaScript variable changes, but React does not know about it
→ UI never updates

Correct: `setCount(count + 1);`
React stores the new value → React schedules a re-render → component re-renders
→ UI shows the new value

React tracks state changes only through the setter function.
Direct modification is invisible to React.

Which component should own the state?

Rule: state belongs to the lowest common ancestor of all components that need it.

Ask:

Does only one component need this data? → State belongs in that component (component state)

Do multiple components need the same data? → State belongs in their common parent (lift state up)

Kažigram example:

posts list – needed by PostList AND PostForm → belongs in App (common parent)

form input text – needed only by PostForm while typing → belongs in PostForm (component state)

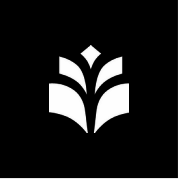
like count – could belong in LikeButton or Post (if needed only by it), or App if persisted

Lifting state up

Problem: PostList and PostForm both need the posts array.
Solution: move the state up to their common parent — App.
- PostForm calls props.onAddPost(newPost) ← event flows up
- App updates state → re-render → PostList gets new posts

```
function App() {
  const [posts, setPosts] = useState([]);   ← state in App

  const addPost = (newPost) => {            ← function in App
    setPosts([...posts, newPost]);
  };
  return (
    <div>
      <PostList posts={posts} />            ← data flows down
      <PostForm onAddPost={addPost} />      ← function flows down
    </div>
  );
}
```



Props — making components configurable
- What are props, and how do they make components configurable?
- How are callback functions passed as props?

State — storing and tracking changing data
- What is state, and how does it differ from props?
- How does useState work, and where should state be owned?

Events and forms — responding to user interaction
- How do React components respond to user interactions?
- How are controlled forms implemented in React?

Side effects and lifecycle — useEffect
- What is the difference between useState and useEffect?
- When and how is useEffect used?

Component libraries — ready-made React components
- What are React component libraries, and how are they used?

Handling events in React

In vanilla JS: `btn.addEventListener('click', handleClick);`
→ event handler added separately

In JSX: `<button onClick={handleClick}>Click me</button>`
→ event handler as a prop, directly in JSX

Common event props: onClick (mouse click), onChange (input value changed), onSubmit (form submitted), onMouseOver (mouse hover). Note: camelCase (onClick, not onclick)

Note: pass the function reference, not a call

- `onClick={handleClick}` ← correct: passes the function
- `onClick={handleClick()}` ← incorrect: calls it immediately

Events update state — React re-renders

The cycle:

1. User clicks the button
2. onClick calls setCount(count + 1)
3. React stores the new value
4. React re-renders the component
5. UI shows the updated count

UI = f(state)
You describe what the UI looks like for any given state. React handles when and what to update.

```
function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}
```

One state update, all dependents re-render

```
// JS
function addPost(newPost) {
  posts.push(newPost);
  render(posts);
}

function render(posts) {
  header.textContent = posts.length + ' posts';
  postList.innerHTML = posts.map(renderCard).join('');
  sidebar.textContent = getMostActive(posts);
}
```

```
// React
function App() {
  const [posts, setPosts] = useState([]);
  return (
    <>
      <Header postCount={posts.length} />
      <PostList posts={posts} />
      <Sidebar posts={posts} />
    </>
  );
}
```

Controlled forms — React owns the input value

In HTML: browser manages the input value internally <input type="text" />
→ user types → browser stores the value
→ you read it only when form is submitted

In React (controlled component): state owns the value
<input value={name} onChange={(e) => setName(e.target.value)} />
→ user types → onChange fires with an event object (e)
→ e.target is the input element → e.target.value is what the user typed
→ setName updates state → React re-renders → input shows new value

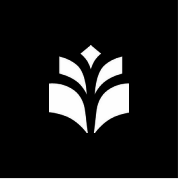
React controls the displayed value — not the browser.
value={name} means: always show what is in state.
Without value={name}, the input is "uncontrolled".

Form with select

```
import { useState } from "react";
export default function CatSelector() {
  const [cat, setCat] = useState("");
  function handleChange(event) {
    setCat(event.target.value);
  }
  return (
    <div>
      {cat ? <p>Selected: {cat}</p> : <p>No cat selected</p>}
      <form>
        <label>
          Choose a cat:
          <select value={cat} onChange={handleChange}>
            <option value="Toast">Toast</option>
            <option value="Naranja">Naranja</option>
          </select>
        </label>
        <input type="submit" value="Submit" />
      </form>
    </div>
  );
}
```

```
// this is equal...
export default function
Forms() { ... }

// ... to this
function Forms() { ... }
export default Forms;
```



Props — making components configurable
- What are props, and how do they make components configurable?
- How are callback functions passed as props?

State — storing and tracking changing data
- What is state, and how does it differ from props?
- How does useState work, and where should state be owned?

Events and forms — responding to user interaction
- How do React components respond to user interactions?
- How are controlled forms implemented in React?

Side effects and lifecycle — useEffect
- What is the difference between useState and useEffect?
- When and how is useEffect used?

Component libraries — ready-made React components
- What are React component libraries, and how are they used?

useState vs. useEffect — different problems, different hooks

useState — storing data that changes
→ "I need to remember a value between renders" → when state changes, component re-renders
example: like count, form input, toggle open/closed

useEffect — reacting to changes or doing work outside React
→ "I need to do something WHEN something changes" or "I need to do something ONCE when component appears", example: update the page title, fetch data, set a timer

useState stores data — React manages the UI
useEffect runs code — at specific moments in the lifecycle

useEffect: for side effects

Side effect = anything that reaches outside React's rendering, e.g. changing the browser (document.title, localStorage), setting up a timer or subscription, or fetching data from a server (example on the slide)

```
function PostList() {
  const [posts, setPosts] = useState([]);  // state for fetched data
                                           // starts as empty array
  useEffect(() => {   // runs after render
    const getData = async () => {
      const res = await fetch('https://api.example.com/posts');
                          // reaches OUTSIDE React o an external server
      const data = await res.json();
      setPosts(data);   // brings data back in, triggers re-render
    };
    getData();
  }, []);             // [] = run only once when component appears
  return posts.map(post =>
    <PostCard key={post.id} title={post.title} />
  );
}
```

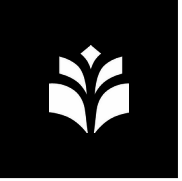
useEffect — the dependency array

```
useEffect(() => {
  // runs after EVERY render
});

useEffect(() => {
  // runs only ONCE — when component first appears
}, []);

useEffect(() => {
  // runs when count changes
}, [count]);
```

in practice: almost always include []
→ prevents infinite loops



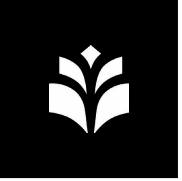
Props — making components configurable
- What are props, and how do they make components configurable?
- How are callback functions passed as props?

State — storing and tracking changing data
- What is state, and how does it differ from props?
- How does useState work, and where should state be owned?

Events and forms — responding to user interaction
- How do React components respond to user interactions?
- How are controlled forms implemented in React?

Side effects and lifecycle — useEffect
- What is the difference between useState and useEffect?
- When and how is useEffect used?

Component libraries — ready-made React components
- What are React component libraries, and how are they used?



Reusable components from libraries

- Same idea as in UI components and JS libraries, but combined
- There are tons of libraries that implement ready-made components for many frequent needs
- e.g. https://github.com/brillout/awesome-react-components libraries organized by category: UI Components, Code Design, Utilities, Performance, Dev Tools, and more

Library: React Bootstrap

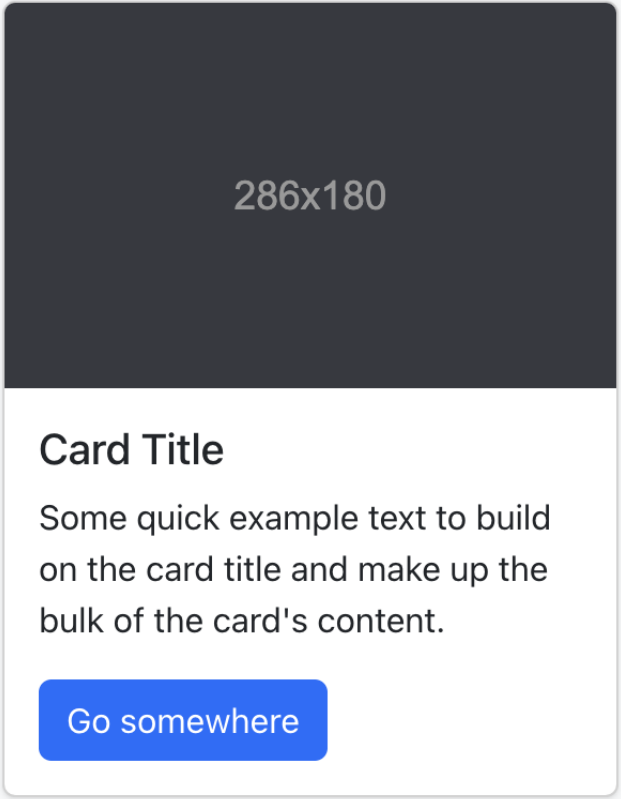
```
npm install react-bootstrap bootstrap
```

https://react-bootstrap.netlify.app/

```jsx
import Button from 'react-bootstrap/Button';
import Card from 'react-bootstrap/Card';

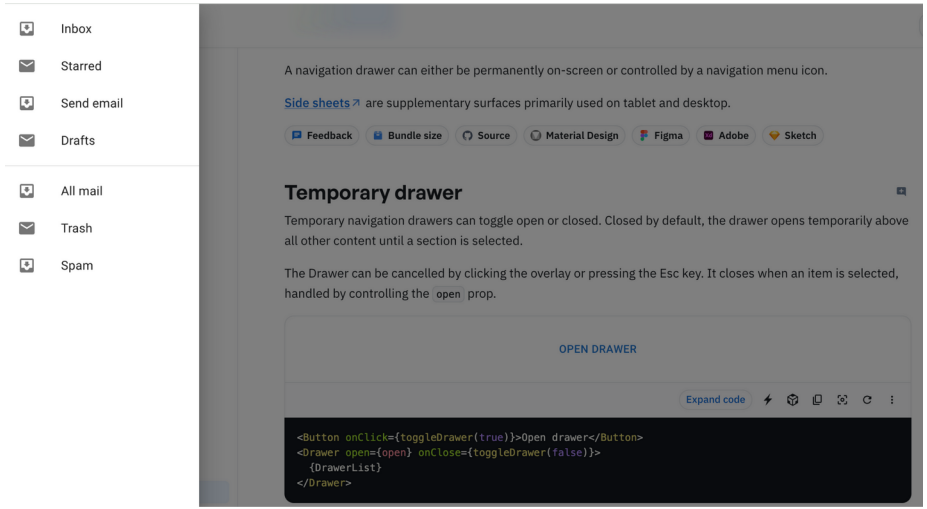
function BasicExample() {
  return (
    <Card style={{ width: '18rem' }}>
      <Card.Img variant="top" src="holder.js/100px180" />
      <Card.Body>
        <Card.Title>Card Title</Card.Title>
        <Card.Text>
          Some quick example text….
        </Card.Text>
        <Button variant="primary">Go somewhere</Button>
      </Card.Body>
    </Card>
  );
}

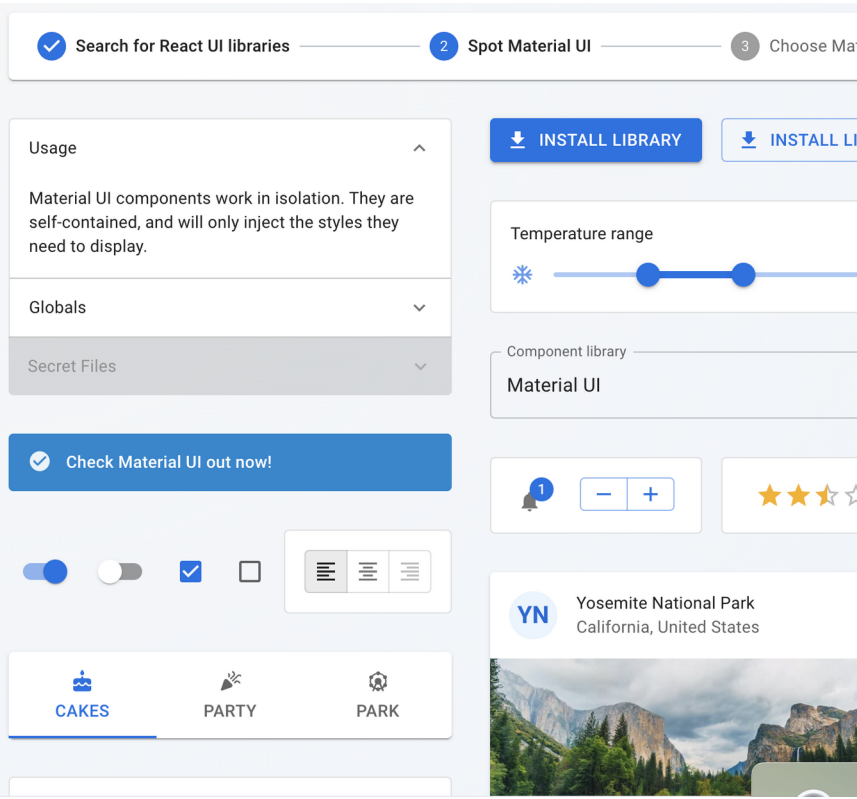
export default BasicExample;
```



Library: Material UI

https://mui.com/material-ui/



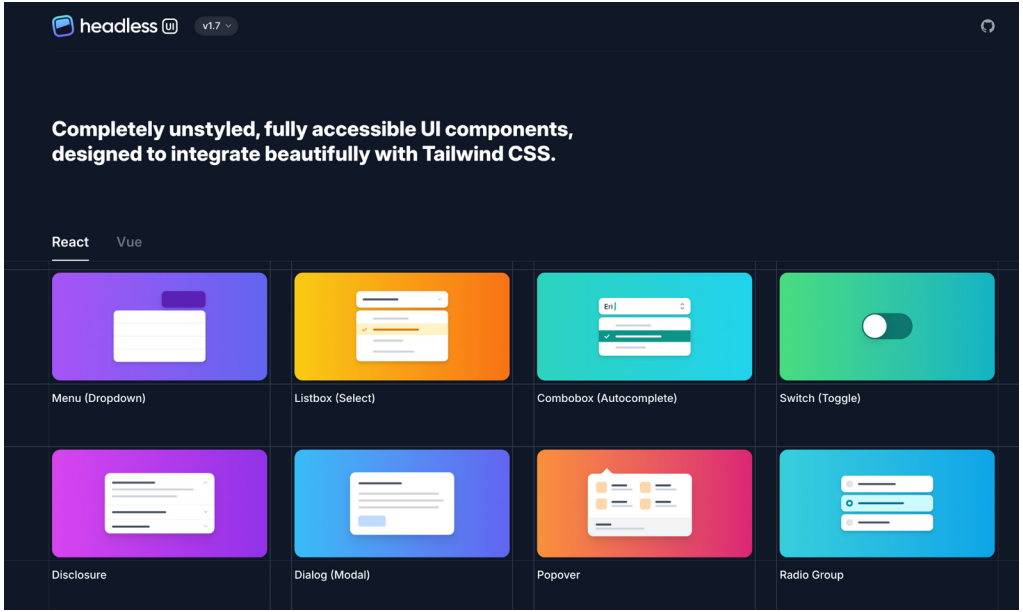


npm install @mui/material @emotion/react @emotion/styled

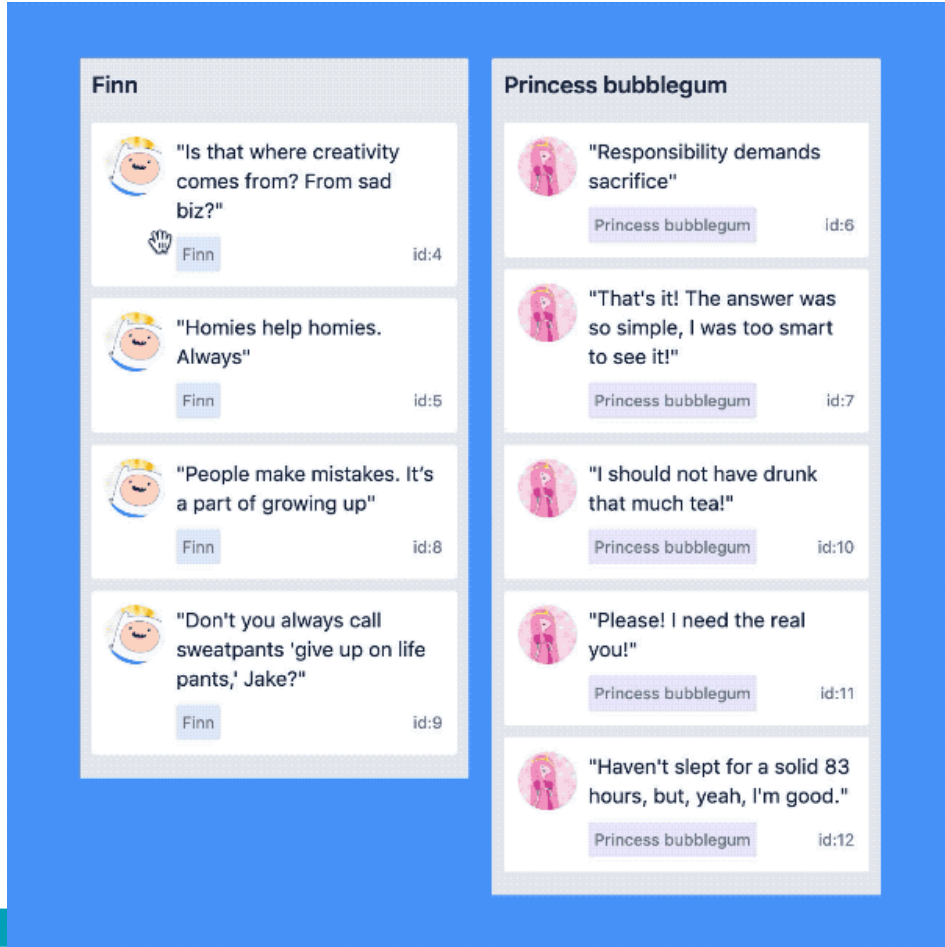
Library: Headless UI

```
npm install @headlessui/react
```

https://headlessui.com/v1



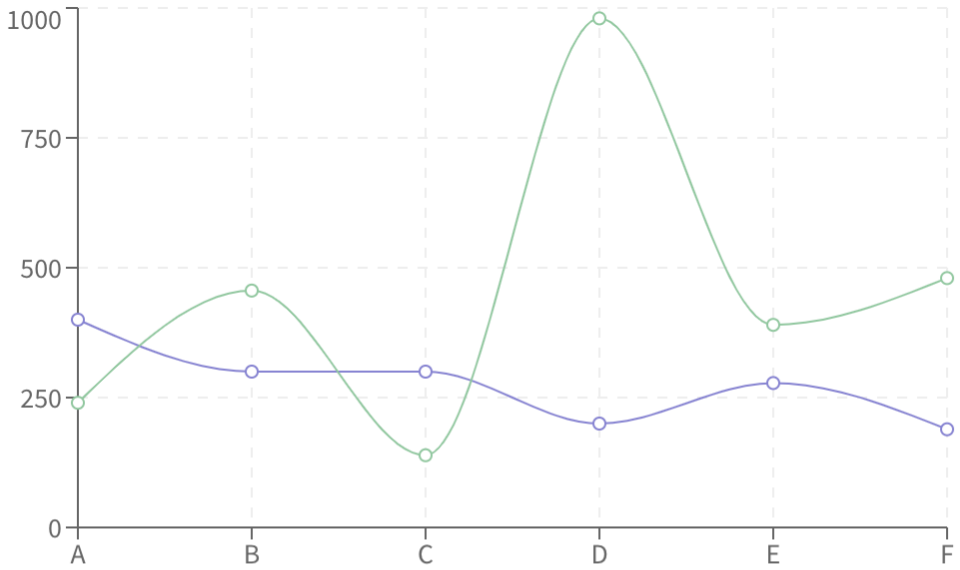
Library: Drag and drop



https://github.com/atlassian/react-beautiful-dnd

Library: Charts (Recharts)

https://recharts.org/




```
<LineChart width={500} height={300} data={data}>
  <XAxis dataKey="name"/>
  <YAxis/>
  <CartesianGrid stroke="#eee" strokeDasharray="5 5"/>
  <Line type="monotone" dataKey="uv" stroke="#8884d8" />
  <Line type="monotone" dataKey="pv" stroke="#82ca9d" />
</LineChart>
```

Library: react-i18next

Internationalization https://react.i18next.com/

- Allows us to define a “dictionary” for all the text in our app so it adapts to the different languages



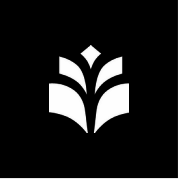
Example: adding a library to React app

1. Install the library with npm: `$ npm install react-bootstrap bootstrap`
2. Import in main.jsx or App.jsx: `import 'bootstrap/dist/css/bootstrap.min.css';`
3. Import and use components.

```jsx
function PostCard({ title, caption }) {
  return (
    <div className="card" style={{ width: '18rem' }}>
      <div className="card-body">
        <h5 className="card-title">{title}</h5>
        <p className="card-text">{caption}</p>
        <button className="btn btn-primary">
          Like
        </button>
      </div>
    </div>
  )
}
```

```jsx
import Button from 'react-bootstrap/Button'
import Card from 'react-bootstrap/Card'

function PostCard({ title, caption }) {
  return (
    <Card style={{ width: '18rem' }}>
      <Card.Body>
        <Card.Title>{title}</Card.Title>
        <Card.Text>{caption}</Card.Text>
        <Button variant="primary">Like</Button>
      </Card.Body>
    </Card>
  )
}
```



Example: GSAP in React

```
// target element by className
import gsap from 'gsap'
const handleLike = () => {
  gsap.fromTo(".like-btn-" + id,
    { scale: 1 },
    { scale: 1.4, duration: 0.15, yoyo: true,
repeat: 1 }
  )
}
<Button className={"like-btn-" + id}
onClick={handleLike}>
  ♡ Like
</Button>
```

+ Simple, uses what we already know (className)
- Less precise, targets by CSS selector, not by direct reference

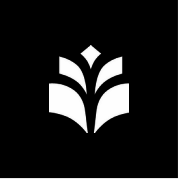
```
// target element by reference to DOM element
import gsap from 'gsap'
import { useRef } from 'react'

const likeRef = useRef(null)

const handleLike = () => {
  gsap.fromTo(likeRef.current,
    { scale: 1 },
    { scale: 1.4, duration: 0.15, yoyo: true,
repeat: 1 }
  )
}

<Button ref={likeRef} onClick={handleLike}>
  ♡ Like
</Button>
```

+ Precise, direct reference to the DOM element
+ React way, works reliably with multiple components



Summary

- Props (properties): read-only values passed parent → child as attributes at render time
- Callback props: functions passed from parent to child, child calls them to send events up (one-way data binding)
- State stores changing data inside a component; differs from props: not passed in, created internally
- useState returns [value, setter] — always update via setter; App state: shared data in common parent, Component state: local UI data only
- Event handlers in JSX as camelCase props: onClick, onChange, onSubmit
- Controlled form: input value={state}, onChange updates setter
- useState stores data; useEffect runs code at specific moments (side effects)
  — main use: fetching data (covered in React III)
- Dependency array controls when useEffect runs: [] = once on mount, [value] = on value change
- Component libraries — ready-made React components, Install with npm install, import components directly, use like custom HTML tags